Taskwarrior Nautical v6 - Quick Start & Cheat Sheet

Use `cp` when the next task depends on completion time. Use `anchor` when it must land on a calendar date. Complete the current task normally; Nautical creates the next link.

For installation details and deeper operational guidance, use the Systems Manual.

---

1. Start in Five Minutes

Install

```
# The managed installer installs the launcher, hooks, and core runtime.
git clone --depth 1 https://github.com/catanadj/taskwarrior-nautical.git
cd taskwarrior-nautical
./nautical install --dry-run
./nautical install

# Register Nautical's Taskwarrior UDAs.
cp uda.conf ~/.task/uda-nautical.conf
grep -Fqx "include $HOME/.task/uda-nautical.conf" "$HOME/.taskrc" 2>/dev/null || \
  printf '\ninclude %s\n' "$HOME/.task/uda-nautical.conf" >> "$HOME/.taskrc"

# Optional: richer panels.
python3 -m pip install -r requirements.txt
```

On Termux, pass `--launcher-path "$PREFIX/bin/nautical"` to the installer.

Create the first chains

```
# Next task is due 3 days after this one is completed.
task add "Follow up" cp:3d due:tomorrow+9h

# First task lands on the next Monday or Friday; Nautical assigns its due date.
task add "Workout" anchor:"w:mon,fri"

# Complete a task normally; Nautical creates its next link.
task +LATEST done
```

If `due` is omitted from an anchor task, Nautical assigns the first matching date automatically.

---

2. Pick the Recurrence Engine

Need	Use	Example
Delay from completion	`cp`	`cp:3d`
Calendar schedule	`anchor`	`anchor:"w:mon,fri"`

Need	Use	Example
Explicit dates from a file	`anchor_file`	`anchor_file:"events.csv"`
Remove calendar dates	`omit` / `omit_file`	`omit:"y:12-25"`

Use `cp` or anchor recurrence on one task, not both.

---

3. Completion-Period Chains (`cp`)

```
cp:3d                    # 3 days after completion
cp:8h                    # 8 hours after completion
cp:"3d,7d,14d"           # repeat a sequence: 3d, then 7d, then 14d
cp:"7d*3,14d"            # repeat 7d three times, then use 14d
cp:"rand(3d..7d)"        # deterministic pick between 3 and 7 days per link
cp:"14d~2d"              # deterministic pick between 12 and 16 days
cp:24h+1s                # exact addition instead of preserving wall-clock time
```

```
# Fixed maintenance interval.
task add "Mow lawn" cp:12d due:tomorrow+9h

# Exact sub-day interval.
task add "Take vitamin" cp:8h due:today+15h

# Repeating treatment sequence.
task add "Insect treatment" cp:"3d,20d,7d,10d,3d" due:today

# Loosely fortnightly inspection.
task add "Routine inspection" cp:"14d~2d" due:today

# Bounded random delay.
task add "Check trap" cp:"rand(3d..7d)" due:today
```

Day-based periods preserve the task's wall-clock due time. Sub-day periods use exact time. Random choices are reproducible per `chainID`, but different chains get independent sequences.

---

4. Anchor Grammar

```
w:mon,fri                     # comma: either Monday or Friday in one atom
w:mon + y:apr                 # plus: Monday that is also in April
w:mon | y:04-12               # pipe: any Monday or April 12
(w:mon | m:last-fri)@t=09:00 # parentheses: group before applying one modifier
w/2:mon                       # /N: every second eligible weekly period
moon:full                     # next local date nearest full moon
w:fri@moon=full               # Friday filtered to full-moon dates
(moon:full + w:fri)@t=20:00 # full moon and Friday at 20:00
```

Moon anchors require optional `astral` and an explicit timezone in `[astronomy.locations.<name>]`. Supported phases are `new`, `first-quarter`,

`full`, and `last-quarter`; aliases such as `full-moon` are accepted. Missing support is reported as an error; Nautical never invents a fallback date.

`+` binds more tightly than `|`. Quote expressions containing spaces, `+`, `|`, or parentheses:

```
task add "April workout" anchor:"w:mon,wed,fri + y:apr"
```

---

5. Weekly Anchors (`w:`)

```
w:mon                # every Monday
w:mon,fri            # every Monday and Friday
w:mon..fri           # every weekday, Monday through Friday
w:fri..mon           # Friday through Monday, wrapping across the week
w:wk                 # weekdays (alias)
w:wd                 # weekdays (alias)
w:we                 # weekend days
w/2:mon              # Monday every second ISO week
w:rand               # one random day in each ISO week
w:2rand              # two distinct random days in each ISO week
```

```
# Monday and Friday workout.
task add "Gym" anchor:"w:mon,fri" due:today+9h

# Monday through Wednesday study block.
task add "Study" anchor:"w:mon..wed"

# Different time on each weekday.
task add "Split training" anchor:"w:mon@t=09:00,fri@t=15:00"

# Two random business days each week.
task add "Field inspection" anchor:"w:2rand@bd@t=09:00"
```

---

6. Monthly Anchors (`m:`)

Calendar and business days

```
m:1                  # first calendar day of each month
m:15                 # 15th calendar day of each month
m:-1                 # last calendar day of each month
m:ld                 # last calendar day (alias)
m:1,15,-1            # first, 15th, and last calendar day
m:1..7               # every date from the 1st through the 7th
m:5bd                # 5th business day of each month
m:-1bd               # last business day of each month
m:lbd                # last business day (alias)
m/2:-1               # last day of every second eligible month
m:rand               # one random date each month
m:3rand              # three distinct random dates each month
```

Weekday positions

```
m:2sat                # second Saturday of each month
m:last-fri            # last Friday of each month
m:1wed,3fri           # first Wednesday and third Friday
m/2:2sat              # second Saturday of every second eligible month
```

```
# First and last day of each month.
task add "Billing sweep" anchor:"m:1,-1" anchor_mode:all due:today

# First day, rolled to the next business day at 09:00.
task add "Payroll" anchor:"m:1@nbd@t=09:00" anchor_mode:all due:today

# Last Friday of each month.
task add "Design session" anchor:"m:last-fri"

# Three random weekdays each month.
task add "Monthly sampling" anchor:"m:3rand + w:mon..fri"
```

---

7. Yearly Anchors (`y:`)

Fixed yearly dates use `MM-DD`.

```
y:05-20                          # every May 20
y:01-15,04-15,07-15,10-15       # January, April, July, and October 15
y:04-20..05-15                   # every date from April 20 through May 15
y:apr                            # every date in April
y:rand                           # one random date each year
y:2rand                          # two distinct random dates each year
y:10-rand                        # one random date in October
y:02-29                          # leap day; years without it are skipped
y:d100                           # 100th calendar day of each year
y:d-1                            # last calendar day of each year
y:d100..d110                     # calendar days 100 through 110
y:w20                            # all seven days of ISO week 20
y:w-1                            # all days of the final ISO week
y:w20 + w:mon                    # Monday of ISO week 20
y:w-1 + w:fri                    # Friday of the final ISO week
y:q1                             # every date in the first quarter
y:q1s                            # every date in January, Q1's start month
y:q1m                            # every date in February, Q1's middle month
y:q1e                            # every date in March, Q1's end month
y/2:w1                           # ISO week 1 every second eligible ISO year
```

Year-day and ISO-week ranges repeat the prefix on both ends: `d100..d110` and `w10..w13`. Negative values count from the end. A nonexistent day 366 or ISO week 53 contributes no date for that year.

```
# Fixed anniversary.
task add "Anniversary" anchor:"y:05-20" anchor_mode:all due:today
```

```
# Two random dates selected only from April, July, or October.
task add "Seasonal audits" anchor:"y:2rand + y:apr,jul,oct"

# Last calendar day at 17:00.
task add "Year-end archive" anchor:"y:d-1@t=17:00"

# Friday of the final ISO week.
task add "ISO year close" anchor:"y:w-1 + w:fri"
```

---

8. Seasonal Anchors

Seasonal selectors use the same positional syntax as week, month, quarter, and year selectors. `season_mode = "fixed"` is the default; set `season_mode = "astronomical"` to derive boundaries from equinoxes and solstices. In astronomical mode, the transition's local calendar date starts the new season.

```
@in-spring                  # Mar 1 through May 31
@in-summer                  # Jun 1 through Aug 31
@in-autumn                  # Sep 1 through Nov 30
@in-winter                  # Dec 1 through Feb 28/29
```

```
# First and last Monday of each spring.
task add "Spring planning" anchor:"(w:mon)@in-spring=first,last"

# Last Friday of each winter at 09:00.
task add "Winter review" anchor:"(w:fri)@in-winter=last@t=09:00"

# First weekday of summer, then one business day later.
task add "Summer handoff" anchor:"(w:mon..fri)@in-summer=first@+1bd"
```

The candidate must be parenthesized. Nautical selects the position inside the season first, then applies modifiers. In fixed mode, winter 2026 starts on December 1, 2026 and ends on February 28, 2027; astronomical mode uses the configured timezone's transition dates instead.

---

9. Positional Selection

Select a date by its position among all candidate matches in a period:

```
(w:mon | w:wed | w:fri)@in-week=last       # last M/W/F match in each week
(w:tue | w:thu)@in-month=first,last         # first and last Tue/Thu each month
(w:mon)@in-quarter=last                     # last Monday of each quarter
(w:mon)@in-year=10th                        # tenth Monday of each year
(w:mon)@in-month=2nd-last                   # second-to-last Monday each month
(w:mon)@in-quarter=last@+1bd                # select, then move one business day
```

Positions accept `first`, `last`, `3rd`, `2nd-last`, and comma-separated lists.
The candidate must be parenthesized and deterministic. Nautical rejects

positions that can never exist and advises when a valid expression is redundant or boundary-sensitive.

```
# Last Monday, Wednesday, or Friday in each week.
task add "Weekly closeout" anchor:"(w:mon | w:wed | w:fri)@in-week=last"

# First and last Tuesday or Thursday in each month.
task add "Twice-monthly review" anchor:"(w:tue | w:thu)@in-month=first,last"

# Tenth Monday of each year at 09:00.
task add "Annual checkpoint" anchor:"(w:mon)@in-year=10th@t=09:00"
```

---

10. Modifiers

```
@t=09:00              # use 09:00 for the matched date
@t=9                  # one-digit hours are accepted (same as 09:00)
@t=09:00,17:30        # create occurrences at both times
@t=06..18/3           # three equally spaced slots: 06:00, 12:00, 18:00
@t=04:30..19:30/3h30min # repeat every 3h30m inside the window
@t=06..12/2h,16..20/2h,22 # compose windows and exact times
@t=rand(06..18)       # one deterministic random minute from 06:00-18:00
@t=rand(06..18/3)     # three random minutes, one from each time bucket
@t=rand(22:30..02:30/3) # overnight random window; late slots are next day
@bd                   # keep the match only when it is a business day
@nbd                  # roll to the next business day
@pbd                  # roll to the previous business day
@nw                   # roll to the nearest weekday
@+2d                  # move two calendar days forward
@-2d                  # move two calendar days backward
@+2bd                 # move two business days forward
@-2bd                 # move two business days backward
@next-mon             # roll forward to the next Monday
@prev-fri             # roll backward to the previous Friday
```

Rolls run first, calendar-day offsets second, and business-day offsets last.
Without `bc`, a business day means Monday through Friday.

```
# Two business days before the rolled month end.
task add "Month-end preparation" anchor:"m:-1@pbd@-2bd"

# Shared time applied to either branch.
task add "Shared review" anchor:"(w:mon | m:last-fri)@t=09:00"

# Previous Friday before December 31.
task add "Year-end reminder" anchor:"y:12-31@prev-fri"
```

---

11. Omit Rules and Date Files

The final schedule is:

```
(anchor + anchor_file dates) - (omit + omit_file dates)
```

```
# Remove every Wednesday from a Monday/Wednesday/Friday schedule.
task add "Workout" anchor:"w:mon,wed,fri" omit:"w:wed"

# Remove Wednesdays only during April, July, and October.
task add "Workout" anchor:"w:mon,wed,fri" omit:"w:wed + y:apr,jul,oct"

# Remove a holiday window.
task add "Workout" anchor:"w:mon,wed,fri" omit:"y:12-24..12-31"
```

Set trusted file directories in `config-nautical.toml`:

```
anchor_file_dir = "/home/user/.task/nautical_anchors"
omit_file_dir = "/home/user/.task/nautical_omits"
```

Plain date file:

```
# One date
2026-01-01
# Inclusive date range
2026-04-20..2026-05-15
# comments are ignored
```

CSV file:

```
date,description
2026-01-01,New Year
2026-12-25,Christmas
```

```
# Use every date in a file.
task add "Company event" anchor_file:"events.csv"

# One day before each file date, at two times.
task add "Event prep" anchor_file:"events.csv@-1d@t=12:00,18:00"

# Weekly schedule excluding file-backed holidays.
task add "Workout" anchor:"w:mon,wed,fri" omit_file:"holidays.csv"

# Use every CSV file in the configured directory.
task add "Calendar review" anchor_file:"*.*"
```

---

12. Named Business Calendars

Define a calendar in `config-nautical.toml`:

```
[business_calendar.work]
anchor = "w:mon..fri"
omit = ["y:01-01", "y:12-25"]
```

```
anchor_file = ["extra-open-days.csv"]
omit_file = ["holidays.csv", "company-closures-*.csv"]
```

Use it through the `bc` UDA:

```
# Last business day according to the work calendar.
task add "Submit payroll" anchor:"m:-1bd@t=16:00" bc:work
```

The selected calendar controls business-day ordinals, filters, rolls, and business-day offsets.

---

13. Modes, Limits, and Stops

```
anchor_mode:skip              # skip missed anchors; default for routines
anchor_mode:all               # create every missed anchor in order
anchor_mode:flex              # skip backlog once, then continue strictly
chainMax:6                    # stop after link 6
chainUntil:2027-12-31         # stop after this local date
chainUntil:2027-12-31T12:00   # stop after this local date and time
until:eod                     # expire this occurrence; the chain continues
until:eod+1s                  # preserve the exact due-to-expiry duration
chain:off                     # disable recurrence manually
```

In `config-nautical.toml`, use `panel_mode = "live"` for a short line-by-line Rich reveal, `live_panel_duration_ms = 0` for no motion, or `panel_mode = "fast"` on slow terminals.

```
# Six Monday/Friday tasks total.
task add "Bootcamp" anchor:"w:mon,fri" chainMax:6 due:today

# Backfill every missed monthly billing occurrence.
task add "Monthly billing" anchor:"m:1" anchor_mode:all due:today

# Stop an existing chain.
task 42 modify chain:off
```

---

14. Recipe Book

Work and administration

```
# Every weekday at 09:00.
task add "Daily review" anchor:"w:mon..fri@t=09:00"

# First business day of every month.
task add "Open monthly books" anchor:"m:1@nbd@t=09:00" anchor_mode:all

# Last business day of every month.
task add "Close monthly books" anchor:"m:-1bd@t=16:00" anchor_mode:all
```

```
# Last Monday of each quarter, moved one business day forward.
task add "Quarter handoff" anchor:"(w:mon)@in-quarter=last@+1bd"

# Four fixed review dates each year.
task add "Quarterly review" anchor:"y:01-15,04-15,07-15,10-15"
```

Maintenance and follow-ups

```
# Twelve days after each completion.
task add "Mow lawn" cp:12d due:tomorrow+9h

# Escalating follow-up sequence, limited to five links.
task add "Client follow-up" cp:"1d,3d,7d,14d,30d" chainMax:5 due:today

# Variable equipment inspection interval.
task add "Inspect equipment" cp:"rand(10d..14d)" due:today

# Last Friday of every month.
task add "Maintenance window" anchor:"m:last-fri@t=18:00"
```

Personal routines

```
# Monday, Wednesday, and Friday at 07:00.
task add "Workout" anchor:"w:mon,wed,fri@t=07:00"

# One random Saturday each month.
task add "Date night" anchor:"m:rand + w:sat"

# Second Saturday each month.
task add "Deep clean" anchor:"m:2sat"

# Last calendar day of the year at 17:00.
task add "Archive journal" anchor:"y:d-1@t=17:00"
```

Random schedules

```
# One random weekday each week.
task add "Surprise practice" anchor:"w:rand@bd"

# Three distinct random weekdays each month.
task add "Field sampling" anchor:"m:3rand + w:mon..fri"

# One random weekend date each month.
task add "Weekend outing" anchor:"m:rand + w:sat,sun"

# Two random dates drawn only from April, July, or October.
task add "Seasonal audits" anchor:"y:2rand + y:apr,jul,oct"
```

---

15. Read-Only Query API

External tools can ask Nautical for authoritative occurrences without importing its internals or reimplementing the scheduler. The command writes one versioned JSON document to stdout and never mutates Taskwarrior:

```
# Matches for one task in a local date range.
nautical query occurrences --uuid TASK_UUID \
  --from 2026-08-24 --to 2026-08-31 | jq

# The next projected occurrence, including chain and daily progress metadata.
nautical query next --uuid TASK_UUID \
  --at 2026-08-24T15:00:00+03:00 | jq

# Discover operations, limits, timestamp formats, and supported providers.
nautical query capabilities | jq
```

Use `status` and structured `failure` fields to distinguish an empty result, invalid recurrence, exhausted search, or unavailable dependency. Returned occurrences include both configured local time and UTC. For skip-mode anchors, `next` may include `lifecycle.daily_instances` with `total`, `current_position`, `missed`, and `upcoming`.

16. Fast Troubleshooting

```
# Read-only installation, configuration, queue, and chain checks.
nautical doctor

# Include machine-readable findings.
nautical doctor --json

# Show hook diagnostics on stderr for one command.
NAUTICAL_DIAG=1 task add "Nautical test" anchor:"w:mon"
```

Check these first:

- `nautical doctor` reports a healthy managed runtime.
- `uda.conf` is included from `~/.taskrc`.
- The task uses either `cp` or anchor recurrence, not both.
- `chainMax`, `chainUntil`, or `chain:off` did not end the chain.
- File-backed dates are inside the configured trusted directory.

Use the Systems Manual for repair, reconciliation, sync, queue, and complete configuration guidance.